

OPERACIÓN HÉLICE NEGRA

Práctica de seguridad ofensiva con Docker — Modalidad individual

CONTEXTO

Eres consultor de seguridad contratado por NebulaCorp, una empresa de análisis de datos meteorológicos. Tras detectar la exfiltración de información sensible de su bóveda interna, el CISO sospecha que un atacante comprometió la infraestructura entrando por su portal corporativo público y pivotando por toda la red interna.

Tu misión es reproducir el camino completo del atacante en un laboratorio controlado, capturar las cinco evidencias (flags) que confirman cada paso del compromiso y documentar el incidente. La infraestructura se levanta como un conjunto de contenedores Docker en tres redes aisladas que simulan la DMZ, la red interna y la red de gestión de la empresa.

REQUISITOS PREVIOS

- Docker y Docker Compose v2 (Desktop o servidor Linux).
- Máquina atacante: Kali Linux (VM, WSL2 o equivalente) con nmap, gobuster/ffuf, curl, hashcat o john, sshpass y sshuttle.
- Wordlist rockyou.txt (incluida en Kali en /usr/share/wordlists/).
- Conocimientos básicos de HTTP, SQL, shell de Linux y SSH.

ARQUITECTURA GENERAL

La infraestructura está repartida en tres redes Docker:

Red	Subred	Alcance
dmz	172.30.10.0/24	Expuesta al host atacante
interna	172.30.20.0/24	Sólo accesible desde dmz
gestion	172.30.30.0/24	Sólo accesible desde interna (marcada internal: true)

Cinco contenedores se reparten por las redes:

Contenedor	Redes	IP	Puerto host	Servicio
web-nebula	dmz + interna	10.10 / 20.10	8080 → 80	nginx + PHP-FPM 8.2
api-nebula	interna	20.20	—	Flask REST 0.4-beta
db-nebula	interna	20.30	—	MariaDB 11.4
bastion-nebula	interna + gestion	20.40 / 30.10	2222 → 22	OpenSSH (Debian)
vault-nebula	gestion	30.20	—	nginx (bóveda)

⚠ La red gestion no es enrutable desde el host. La única forma de hablar con vault-nebula es a través del bastión. Esto es deliberado: el último salto del compromiso es la prueba de que has pivotado correctamente.

DESPLIEGUE DEL LABORATORIO

Desde el directorio `infra/` que acompaña a esta práctica:

```
$ docker compose up -d --build
$ docker compose ps          # debe mostrar los 5 contenedores 'running'
$ curl -s http://localhost:8080/ | head -5
```

✓ Verificación: la página del portal de NebulaCorp se sirve en `http://localhost:8080/` y SSH del bastión escucha en `localhost:2222`.

FASES DE LA PRÁCTICA

FASE 1 — RECONOCIMIENTO Y ENUMERACIÓN

Desde tu máquina atacante, identifica qué puertos y servicios expone NebulaCorp al exterior. Enumera el servidor web en busca de rutas, ficheros de respaldo y configuraciones expuestas por error.

- Escaneo de puertos del host con nmap (incluyendo banner grabbing).
- Enumeración web con gobuster o ffuf sobre el portal del puerto 8080.
- Inspección manual de la página, sus enlaces y el código HTML servido.

✓ Verificación: deberías localizar la primera flag dentro de un fichero expuesto en un directorio de respaldo que el equipo de desarrollo olvidó blindar. La flag tiene formato `HELICE{...}`.

FASE 2 — INCLUSIÓN LOCAL DE FICHEROS (LFI)

El visor de páginas del portal (`view.php`) recibe un parámetro `page` sin saneamiento. Identifica el patrón vulnerable y abusa de él para leer archivos arbitrarios del servidor.

- Lee `/etc/passwd` para confirmar la vulnerabilidad y enumerar usuarios.
- Localiza el fichero `.env` del docroot y extrae las credenciales y URLs internas que contiene.
- Recupera la segunda flag desde un fichero dentro de un directorio que el servidor web no lista pero que sí se puede leer mediante la vulnerabilidad.

✓ Verificación: debes obtener la clave de API de la aplicación (`API_KEY`) y la URL del servicio REST interno. Ambas serán necesarias en la Fase 4.

FASE 3 — EJECUCIÓN REMOTA DE CÓDIGO POR ENVENENAMIENTO DE LOGS

La LFI por sí sola sólo permite leer ficheros. Combínala con el envenenamiento del log de acceso de nginx para ejecutar comandos arbitrarios dentro del contenedor web.

- Inyecta un payload PHP en una cabecera HTTP que nginx registre en su log de acceso.
- Incluye el log mediante la LFI y dispara la ejecución del código inyectado pasando el comando como parámetro adicional.
- Demuestra la RCE leyendo la tercera flag, alojada fuera del docroot.

⚠ Cuidado con las comillas: nginx escapa las comillas dobles que aparecen en las cabeceras a la secuencia literal `\x22`, lo que rompe el parseo del código PHP cuando se incluye el log. Si te equivocas y envenenas el log con un payload defectuoso, todas las peticiones posteriores fallarán con un error fatal de PHP y deberás reiniciar el contenedor.

✓ Verificación: ejecutas comandos como `www-data` dentro del contenedor `web-nebula`. Desde ahí puedes resolver los nombres `api-nebula`, `db-nebula` y `bastion-nebula` porque el contenedor también pertenece a la red interna.

FASE 4 — INYECCIÓN SQL CONTRA EL API INTERNO

Con la RCE en el contenedor web, ya tienes pivote hacia la red interna. El API REST de catálogo de productos sólo acepta peticiones desde dentro y exige una cabecera X-API-Key, que ya conseguiste en la Fase 2.

- Consume el endpoint `/products?q=...` pasando la cabecera de autenticación.
- Identifica el patrón vulnerable y monta un UNION SELECT que respete el número y tipo de columnas devueltas.
- Extrae el contenido de las tablas users (hashes) y secrets (flag4 + pistas).

✓ *Verificación: obtienes los hashes MD5 de los usuarios — al menos uno se craquea fácilmente con rockyou.txt. La cuarta flag aparece en la tabla secrets.*

FASE 5 — MOVIMIENTO LATERAL, ESCALADA Y BÓVEDA FINAL

Con los hashes en la mano, craquea al menos el de un usuario cuyo rol haga plausible que tenga acceso al bastión. El bastión SSH sí está expuesto al host (puerto 2222), pero sus credenciales viven en la BBDD que sólo conseguiste leer porque comprometiste antes el portal.

- Crackea el hash MD5 con hashcat (modo 0) o john usando rockyou.txt.
- Entra por SSH al bastión y lee el fichero de bienvenida que deja el equipo de sistemas — contiene una pista crítica.
- El endpoint `/admin/dump` de vault-nebula requiere un token Bearer; el token está en `/root/.vault_token`, sólo legible por root.
- Escala privilegios aprovechando la configuración de sudo vigente (consulta GTFOBins) y extrae el token.
- Desde el bastión, consume el endpoint protegido de la bóveda con el token y captura la quinta flag.

⚠ *La red gestion donde vive vault-nebula está marcada como internal en Docker. Cualquier intento de hacer curl `http://vault-nebula` desde tu máquina atacante o desde el contenedor web fallará — sólo el bastión tiene una interfaz dentro de esa red.*

PREGUNTAS TEÓRICAS PARA RESPONDER

Entrega las respuestas por escrito junto a las flags.

Pregunta A — Describe paso a paso el camino del atacante desde su máquina hasta la bóveda. Para cada salto indica: contenedor origen, contenedor destino, red Docker que atraviesa, puerto destino, técnica empleada (LFI, RCE, SQLi, etc.) y artefacto que extrae.

Pregunta B — Sin cambiar la arquitectura de tres redes, ¿qué controles concretos a nivel de aplicación (web-nebula y api-nebula) habrían impedido las fases 2, 3 y 4? Da al menos una contramedida específica por cada fase, con el parámetro de configuración o el cambio de código que implica.

Pregunta C — La red gestion está marcada como internal: true. ¿Qué consigue eso exactamente, y por qué no es suficiente como única defensa? Sustenta la respuesta con lo que has observado durante la Fase 5.

Pregunta D — Para el envenenamiento de logs de la Fase 3, explica por qué la inyección con comillas dobles falla pero con comillas simples tiene éxito. Razónalo describiendo qué hace nginx con las cabeceras y cómo eso interactúa con el parser de PHP cuando se incluye el log.

NOTAS IMPORTANTES

- Todas las flags tienen formato HELICE{...}. Apunta cada una con la fase a la que pertenece.
- Reiniciar el laboratorio entre intentos: `docker compose down -v && docker compose up -d --build`.
- El portal NO debe ser accesible más allá de tu host. Si tu máquina está en una red compartida, levanta el laboratorio dentro de una VM aislada.
- Las vulnerabilidades de este laboratorio son intencionadas y conocidas. NO repliques estas configuraciones en sistemas reales.

CRITERIOS DE EVALUACIÓN

Fase	Puntos
Fase 1 — Reconocimiento + flag1	10
Fase 2 — LFI + flag2 + credenciales del .env	15
Fase 3 — RCE por log poisoning + flag3	20
Fase 4 — SQLi en API interna + flag4	25
Fase 5 — Lateral + privesc + bóveda + flag5	20
Preguntas teóricas (A, B, C, D)	10
TOTAL	100